



Security Audit

Vrine “Main Contracts” (Blockchain)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	35
Conclusion	36
Our Methodology	37
Disclaimers	39
About Hashlock	40

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Vrine team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Vrine Network is an innovative no-code platform that allows users to create, launch, scale, and monetize blockchain-based projects without writing a single line of code. Its mission is to democratize access to blockchain technology, enabling entrepreneurs, businesses, and enthusiasts to develop decentralized solutions in an accessible and efficient way. The platform provides an intuitive interface, integrated tools, and support for features such as smart contracts and tokens, making it easier to implement innovative ideas within the Web3 ecosystem.

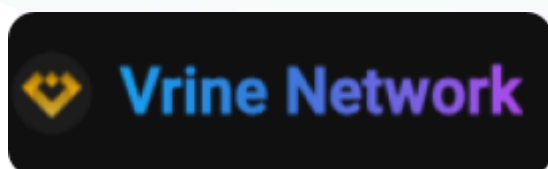
Project Name: Vrine

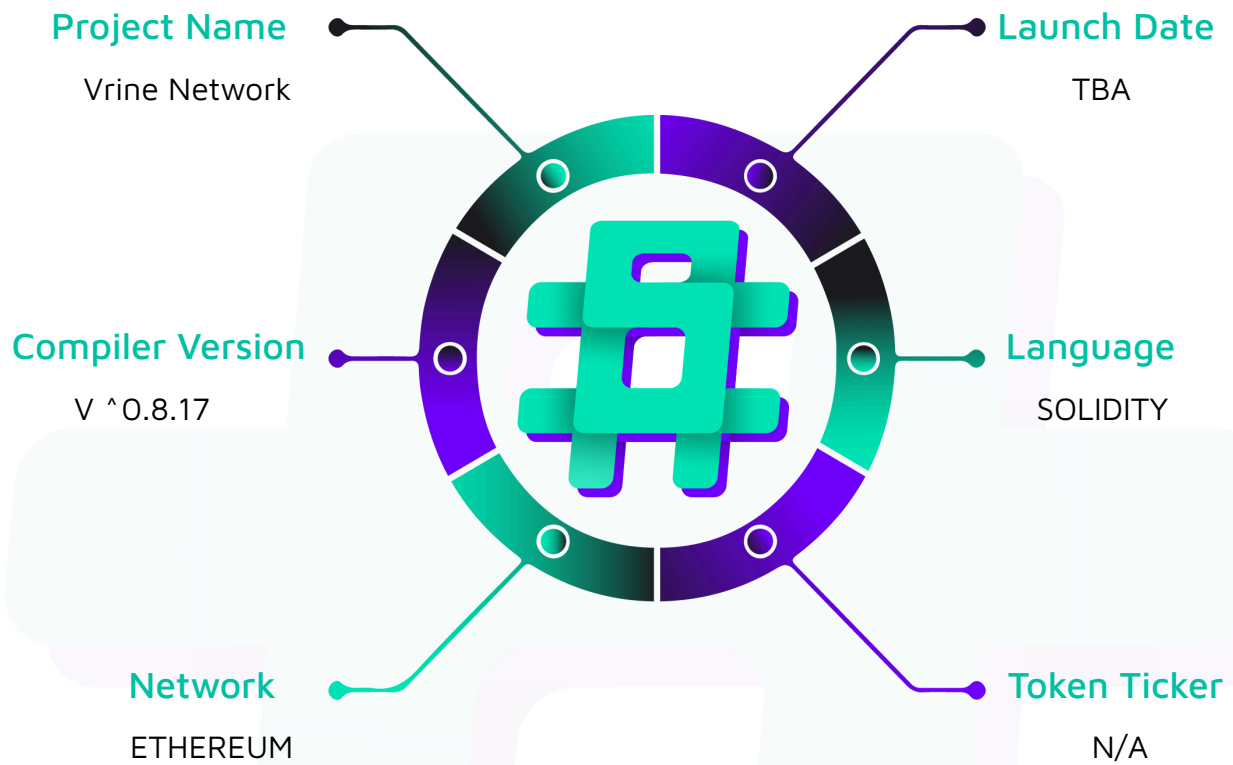
Project Type: Blockchain

Compiler Version: ^0.8.17

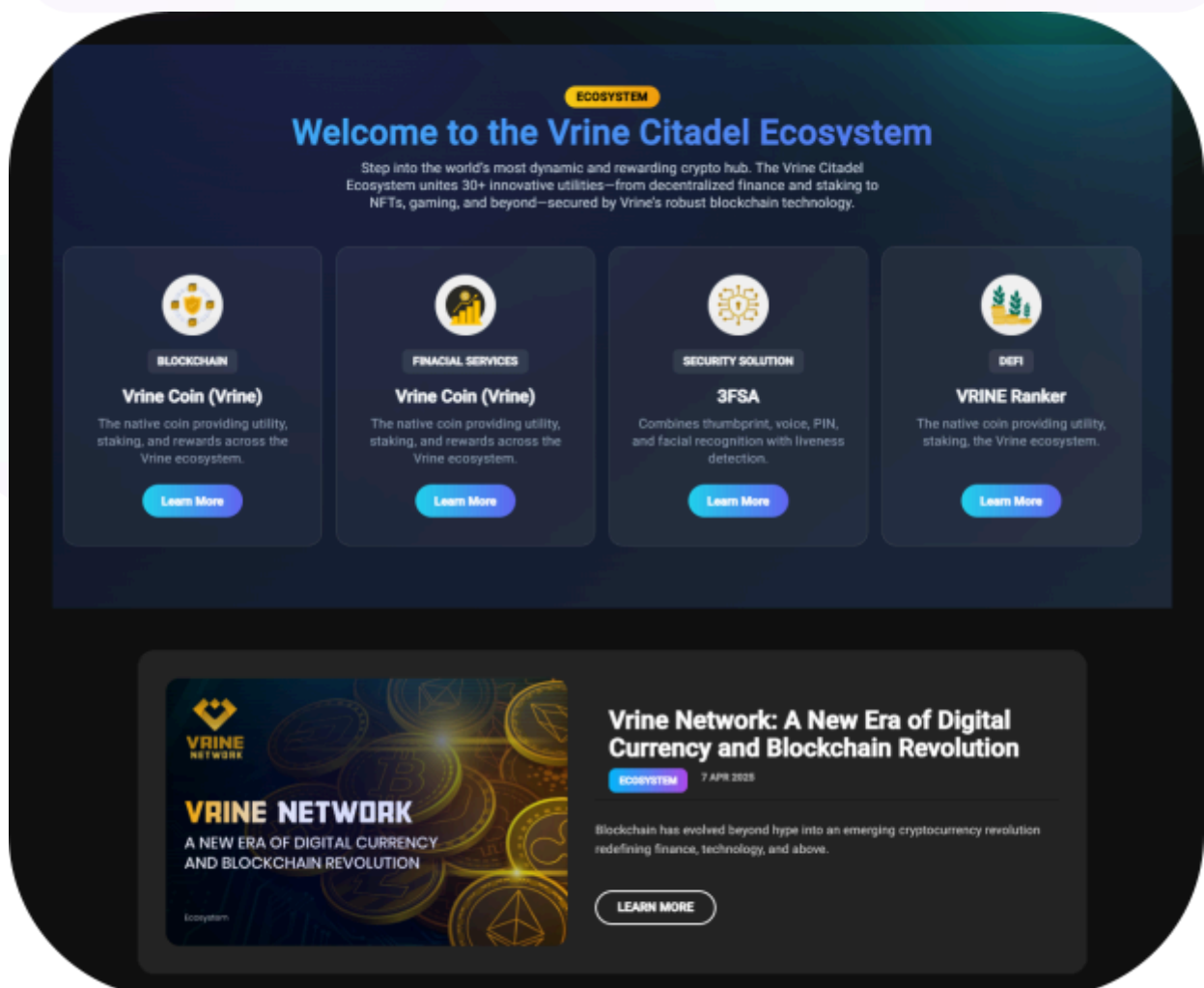
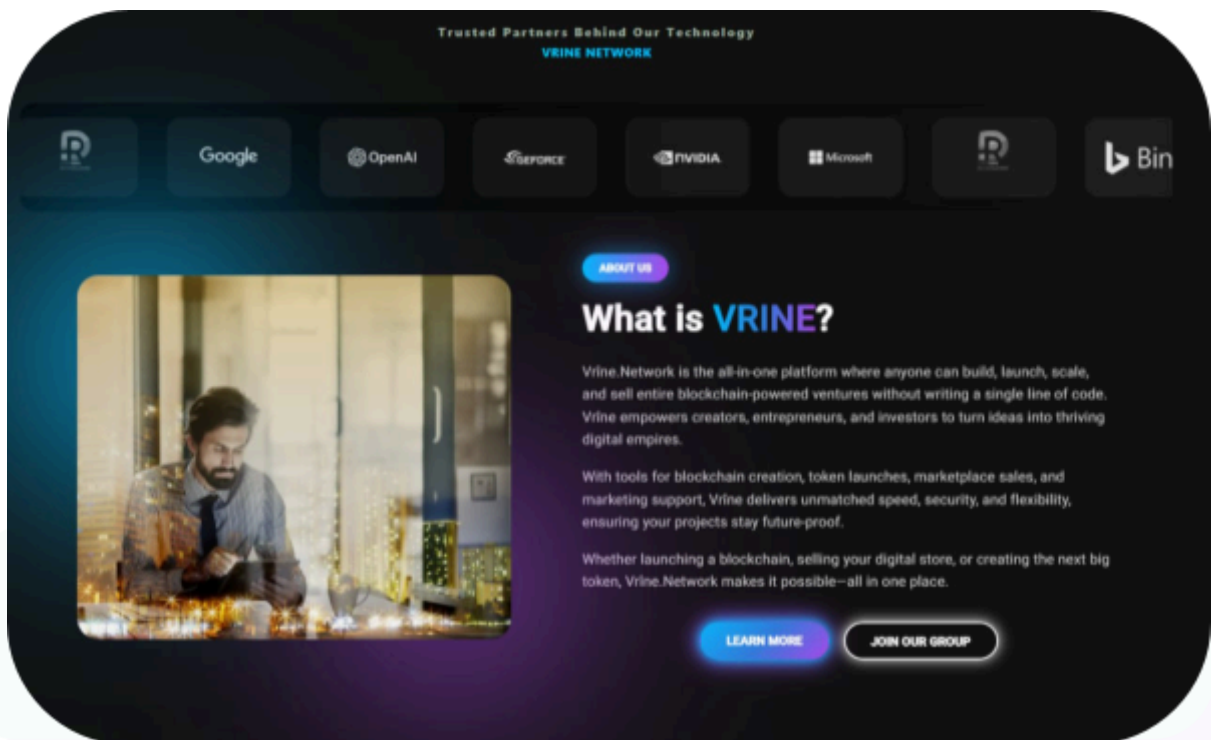
Website: <https://vrine.network/>

Logo:



Visualised Context:

Project Visuals:



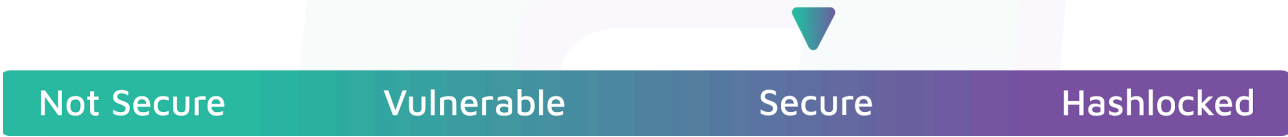
Audit Scope

We at Hashlock audited the solidity code within the Vrine Network project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Vrine Network Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	February, 2026
Contract 1	Vrine.sol
Contract 2	VrineDividendTracker.sol
Contract 3	DividendPayingToken.sol
Contract 4 (Interface)	DividendPayingTokenInterface.sol
Contract 5 (Interface)	DividendPayingTokenOptionalInterface.sol

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts implement a reflection-based dividend token with fee-on-transfer mechanics and automated swap functionality. They utilize well-established patterns from OpenZeppelin (Ownable, ERC20, SafeMath) and integrate with Uniswap V2 for swaps and liquidity provisioning. After Hashlock's Audit, we found the smart contracts to be "Vulnerable".



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

2 High severity vulnerabilities

2 Medium severity vulnerabilities

4 Low severity vulnerabilities

5 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Vrine.sol - Allows the owner to: - Enable/disable trading - Configure buy/sell fee parameters - Set max buy/sell/wallet limits - Exclude addresses from fees and limits - Blacklist bot addresses - Update router and pair addresses - Update dividend tracker - Rescue ETH and foreign tokens - Enable/disable burn functionality - Allows users to: - Transfer tokens (with automatic fee deduction) - Buy and sell on Uniswap V2 - Burn tokens (when enabled) - Claim USDC dividends - Automated functions: - Collect fees on buys/sells - Swap accumulated fees to ETH/USDC - Add liquidity to Uniswap - Distribute USDC dividends to tracker	Contract achieves this functionality.
VrineDividendTracker.sol - Allows the owner (Vrine contract) to: - Set holder balances for dividend calculation - Process dividend claims	Contract achieves this functionality.

- Exclude addresses from dividends - Update claim wait period - Allows privileged address to: - Rescue ETH and foreign tokens - Set minimum token balance for eligibility	
DividendPayingToken.sol - Allows the owner to: - Distribute USDC dividends to holders - Provides internal functions for: - Minting/burning tracker tokens - Setting holder balances - Calculating dividend entitlements using magnified dividend math - Allows users to: - Withdraw accrued dividends - View withdrawable, withdrawn and accumulative dividend amounts	Contract achieves this functionality.
DividendPayingTokenInterface.sol Interface defining required functions for dividend-paying tokens: - dividendOf() - view claimable dividends - withdrawDividend() - withdraw dividends Defines events: - DividendsDistributed - DividendWithdrawn	Interface only - no executable logic. Correctly implemented by DividendPayingToken.sol.
DividendPayingTokenOptionalInterface.sol Interface defining optional view functions: - withdrawableDividendOf() - view pending dividends - withdrawnDividendOf() - view claimed dividends - accumulativeDividendOf() - view total earned dividends	Interface only - no executable logic. Correctly implemented by DividendPayingToken.sol.

Code Quality

This audit scope involves the smart contracts of the Vrine Network project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Vrine Network project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

The smart contracts utilize well-established open-source patterns from OpenZeppelin (Context, Ownable, ERC20, SafeMath) and Roger Wu's dividend-paying token implementation. External interactions are limited to Uniswap V2 protocol (Router, Factory, Pair) and standard ERC20 token calls (USDC).

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] VrineDividendTracker - Hardcoded EOA can drain all dividend funds and manipulate eligibility

Description

VrineDividendTracker smart contract contains privileged functions gated to a hardcoded EOA instead of onlyOwner. That address can drain all USDC/ETH/tokens from the dividend pool and arbitrarily change dividend eligibility thresholds.

Vulnerability Details

The following functions bypass ownership and allow a fixed external address to control all tracker funds and configuration:

```
function rescueForeignTokensDiv(address tkn) public {  
  
    require(msg.sender == 0x2ce66Ba02757373131954D40E66ef01AA081Eb8f, "Ownable: caller  
is not the owner");  
  
    uint amount = IERC20(tkn).balanceOf(address(this));  
  
    IERC20(tkn).transfer(msg.sender, amount);  
  
}  
  
function rescueETHDiv() public {  
  
    require(msg.sender == 0x2ce66Ba02757373131954D40E66ef01AA081Eb8f, "Ownable: caller  
is not the owner");  
  
    bool success;  
  
    (success,) = address(msg.sender).call{value: address(this).balance}("");  
  
}
```

```
function setminimumTokenBalanceForDividends(uint256 _newAmt) public {  
    require(msg.sender == 0x2ce66Ba02757373131954D40E66ef01AA081Eb8f, "Ownable: caller  
is not the owner");  
    minimumTokenBalanceForDividends=_newAmt;  
}
```

This address can withdraw the entire USDC dividend pool at any time and change eligibility rules (e.g., exclude all holders).

Impact

- Total loss of all dividend funds in the tracker
- Arbitrary manipulation of who receives dividends
- Permanent centralization / rug capability

Recommendation

Remove the hardcoded EOA and gate these functions with `onlyOwner` or a documented admin role. Avoid fixed privileged addresses.

Status

Resolved

[H-02] Vrine#burn() - Burn does not update dividend tracker, allowing over-claim of dividends

Description

The `Vrine#burn()` function reduces a holder's token balance, but does not update the dividend tracker balance. A burner can retain full dividend entitlement despite holding fewer tokens. There is one precondition for this issue - the owner must have enabled burning via `FlipisBurnallow(true)`. By default, `isBurnallow = false`.

Vulnerability Details

`burn()` destroys tokens but never syncs the tracker:

```
function burn(uint256 value) external virtual {  
  
    require(isBurnallow == true, "Burn is not allowed!");  
  
    _burn(_msgSender(), value);  
  
    emit TokensBurned(_msgSender(), value);  
  
    // MISSING: dividendTracker.setBalance(payable(msg.sender), balanceOf(msg.sender));  
  
}
```

Dividend entitlement is based on balances stored in the tracker (`DividendPayingToken`). Without calling `dividendTracker.setBalance()`, the tracker continues to treat the user as holding the pre-burn amount.

Compare to `_transfer()` which properly syncs:

```
function _transfer(address from, address to, uint256 amount) internal override {  
  
    // ... code ...  
  
    super._transfer(from, to, amount);  
  
    try dividendTracker.setBalance(payable(from), balanceOf(from)) {} catch {}  
  
    try dividendTracker.setBalance(payable(to), balanceOf(to)) {} catch {}  
  
    // ... code ...  
  
}
```


Exploit Flow

1. User holds 1,000,000 VRINE tokens
2. Dividend tracker balance: 1,000,000
3. Owner enables burning: `FlipisBurnallow(true)`
4. User calls `burn(999,999)`
5. User's VRINE balance: 1 token
6. User's dividend tracker balance: still 1,000,000 (never updated)
7. New dividends are distributed, `magnifiedDividendPerShare` increases
8. User calls `claim()` on Vrine contract
9. `accumulativeDividendOf()` calculates based on tracker balance (1,000,000), not actual balance (1)
10. User receives dividends as if holding 1,000,000 tokens while only holding 1

Impact

- Burner keeps a full dividend share after reducing holdings
- Other holders receive diluted dividends
- Direct value extraction from the dividend pool

Recommendation

After burning, sync the tracker.

```
function burn(uint256 value) external virtual {
    require(isBurnallow == true, "Burn is not allowed!");
    _burn(_msgSender(), value);
    try dividendTracker.setBalance(payable(_msgSender()), balanceOf(_msgSender())) {}
catch {}
    emit TokensBurned(_msgSender(), value);
}
```

Status

Resolved

Medium

[M-01] Vrine#swapTokensForEth(), Vrine#swapTokensForUSDC() and Vrine#addLiquidity() - Zero slippage on swaps and liquidity adds enables MEV extraction

Description

All DEX interactions use zero minimum output. MEV bots can sandwich protocol swaps and liquidity adds, extracting value from accumulated fees.

Vulnerability Details

All three swap and liquidity functions set minimums to zero:

```
function swapTokensForEth(uint256 tokenAmount) private {  
  
    address[] memory path = new address[](2);  
  
    path[0] = address(this);  
  
    path[1] = uniswapV2Router.WETH();  
  
  
    _approve(address(this), address(uniswapV2Router), tokenAmount);  
  
  
    uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransferTokens(  
  
        tokenAmount,  
  
        0, // accept any amount of ETH  
  
        path,  
  
        address(this),  
  
        block.timestamp  
  
    );  
  
}
```

```

function swapTokensForUSDC(uint256 tokenAmount) private {

    address[] memory path = new address[](3);

    path[0] = address(this);

    path[1] = uniswapV2Router.WETH();

    path[2] = USDC;

    _approve(address(this), address(uniswapV2Router), tokenAmount);

    uniswapV2Router.swapExactTokensForTokensSupportingFeeOnTransferTokens(

        tokenAmount,

        0, // accept any amount of USDC

        path,

        address(this),

        block.timestamp

    );
}

function addLiquidity(uint256 tokenAmount, uint256 ethAmount) private {

    _approve(address(this), address(uniswapV2Router), tokenAmount);

    uniswapV2Router.addLiquidityETH({value: ethAmount}(

        address(this),

        tokenAmount,

        0, // amountTokenMin = 0

        0, // amountETHMin = 0

        _liqWallet,

```

```
        block.timestamp  
  
    );  
  
}
```

Whenever `_transfer()` triggers the swap cycle, the transaction is visible in the mempool. Bots can move price before execution and reverse after, capturing the difference.

Impact

- Continuous leakage of protocol fee value
- Reduced ETH/USDC delivered to dividends and wallets
- Suboptimal liquidity provisioning

Recommendation

Use slippage protection based on on-chain quotes with a tolerance threshold (e.g., $\text{expectedOut} \times \text{tolerance}$). Revert if received amounts fall below bounds.

Status

Resolved

[M-02] `Vrine#swapAndSendToFee()` - Division by zero even under valid fee configuration in Vrine smart contract

Description

If marketing, dev, and charity fees are all set to zero while other fees (rewards, liquidity) remain non-zero, `swapAndSendToFee()` will divide by zero and revert. This configuration passes all validation checks in the fee setter functions, making it a valid but dangerous configuration that permanently bricks all swap-triggering transfers.

Vulnerability Details

The fee setter functions allow setting marketing, dev and charity to zero while keeping other fees non-zero:

```
function buyFee(uint256 _BUY_USDCRewardsFee, uint256 _BUY_liquidityFee, uint256
_BUY_marketingFee, uint256 _BUY_CharityFee, uint256 _BUY_devFee) public onlyOwner {

    require(_BUY_USDCRewardsFee.add(_BUY_liquidityFee).add(_BUY_marketingFee).add(_BUY
_CharityFee).add(_BUY_devFee) <= 100, "Must keep buy taxes below 10%");

    BUY_USDCRewardsFee = _BUY_USDCRewardsFee;

    BUY_liquidityFee = _BUY_liquidityFee;

    BUY_marketingFee = _BUY_marketingFee;

    BUY_CharityFee = _BUY_CharityFee;

    BUY_devFee = _BUY_devFee;

}
```

Example valid configuration that triggers the bug:

```
buyFee(20, 10, 0, 0, 0); // rewards=2%, liquidity=1%, marketing=0, charity=0, dev=0

// Sum = 30 ≤ 100 ✓ passes validation
```

When a swap is triggered, `swapAndSendToFee()` attempts to split ETH between wallets:

```
function swapAndSendToFee(uint256 tokens) private {

    uint256 initialBalance = address(this).balance;
```

```

swapTokensForEth(tokens);

uint256 newBalance = address(this).balance.sub(initialBalance);

if (_totalFees > 0) {
    uint256 marketingAmount = _marketingFee * newBalance / (_marketingFee + _devFee +
_charityFee);

    uint256 charityAmount = _charityFee * newBalance / (_marketingFee + _devFee +
_charityFee);

    uint256 devWalletAmount = newBalance - marketingAmount - charityAmount;

    if (marketingAmount > 0) {
        (bool mainmarketingTransferSuccess, ) = _marketingWalletAddress.call{value:
marketingAmount}("");

        require(mainmarketingTransferSuccess, "Marketing wallet transfer failed");
    }

    if (charityAmount > 0) {
        (bool maincharityTransferSuccess, ) = _charityWalletAddress.call{value:
charityAmount}("");

        require(maincharityTransferSuccess, "Charity wallet transfer failed");
    }

    if (devWalletAmount > 0) {
        (bool devWalletTransferSuccess, ) = _devWalletAddress.call{value:
devWalletAmount}("");

        require(devWalletTransferSuccess, "Dev wallet transfer failed");
    }
}

```

```

    }
}

```

The condition `if (_totalFees > 0)` passes because `_totalFees = 30` (rewards + liquidity), but the denominator `(_marketingFee + _devFee + _charityFee) = 0`. In result, the whole transaction reverts.

Trigger Flow

1. Owner calls `buyFee(20, 10, 0, 0, 0)` - passes validation
2. User performs a buy, so `_setBuy()` sets `_marketingFee = _devFee = _charityFee = 0`
3. Contract accumulates tokens above `swapTokensAtAmount`
4. Next sell triggers swap cycle
5. `swapAndSendToFee()` executes with zero denominator
6. Transaction reverts with `Panic(0x12)`
7. All future swap-triggering transfers permanently fail

Impact

- Swap cycle reverts
- Transfers that trigger swaps fail
- Fee distribution halted

Recommendation

Add validation in fee setters to ensure the denominator cannot be zero when other fees exist or guard the denominator in `swapAndSendToFee()`.

Status

Resolved

Low

[L-01]

Vrine.sol#updateUniswapV2Router()/Vrine.sol#updateuniswapV2Pair()

-

Router/Pair update does not register AMM pair or dividend exclusions

Description

The router and pair update functions change `uniswapV2Pair` but do not update the AMM registry (`automatedMarketMakerPairs`) or dividend exclusions. This diverges from constructor behavior and leaves pair-dependent logic inconsistent after migration.

Vulnerability Details

Constructor correctly wires the pair:

```
address _uniswapV2Pair = IUniswapV2Factory(_uniswapV2Router.factory())
    .createPair(address(this), _uniswapV2Router.WETH());

_setAutomatedMarketMakerPair(_uniswapV2Pair, true);
```

But update functions only set the variable:

```
function updateUniswapV2Router(address newAddress) external onlyOwner {
    uniswapV2Router = IUniswapV2Router02(newAddress);

    address _uniswapV2Pair = IUniswapV2Factory(uniswapV2Router.factory())
        .createPair(address(this), uniswapV2Router.WETH());

    uniswapV2Pair = _uniswapV2Pair; // no AMM registration
}

function updateuniswapV2Pair(address _newuniswapV2Pair) external onlyOwner {
    uniswapV2Pair = _newuniswapV2Pair; // no AMM registration
}
```


No `_setAutomatedMarketMakerPair()` or dividend exclusion is applied.

Impact

After router/pair update:

- AMM detection becomes incorrect
- swap gating may trigger at wrong times
- LP pair can receive dividends
- old pair remains flagged

Protocol behavior becomes inconsistent after migration.

Recommendation

Mirror constructor wiring in both update functions:

```
_setAutomatedMarketMakerPair(_newPair, true);  
dividendTracker.excludeFromDividends(_newPair);
```

Status

Resolved

[L-02] Vrine.sol#updateDividendTracker() - New dividend tracker misses exclusions for deadWallet and uniswapV2Pair

Description

When replacing the dividend tracker, the function does not exclude `deadWallet` and `uniswapV2Pair`, even though both are excluded in the constructor.

Vulnerability Details

Constructor exclusions:

```
dividendTracker.excludeFromDividends(deadWallet);  
  
// pair excluded via _setAutomatedMarketMakerPair
```

Update path:

```
newDividendTracker.excludeFromDividends(address(newDividendTracker));  
  
newDividendTracker.excludeFromDividends(address(this));  
  
newDividendTracker.excludeFromDividends(owner());  
  
newDividendTracker.excludeFromDividends(address(uniswapV2Router));  
  
// Missing: deadWallet, uniswapV2Pair
```

Impact

After tracker replacement:

- dead wallet may accrue dividends
- LP pair may accrue dividends
- legitimate holders are diluted

Recommendation

Add missing exclusions:

```
newDividendTracker.excludeFromDividends(deadWallet);  
  
newDividendTracker.excludeFromDividends(uniswapV2Pair);
```

Status

Resolved

[L-03] `Vrine.sol#rescueETH()` - ETH rescue ignores transfer success and can fail silently

Description

`rescueETH()` performs a low-level ETH transfer but ignores the success flag, allowing silent failure.

Vulnerability Details

The `success` variable is unused in `rescueETH()`.

```
function rescueETH() public onlyOwner {  
    bool success;  
    (success,) = address(msg.sender).call{value: address(this).balance}("");  
}
```

Impact

The owner may believe ETH was rescued when it was not.

Recommendation

Add a success check:

```
(bool success,) = msg.sender.call{value: address(this).balance}("");  
require(success, "ETH transfer failed");
```

Status

Resolved

[L-04] **Vrine.sol#sendETHToFee()** - Fee payout omits charity wallet and uses `.transfer()` (2300 gas stipend)

Description

`sendETHToFee()` distributes ETH only to marketing and dev wallets, omitting the charity wallet. It also uses `.transfer()`, which imposes a 2300 gas limit and may fail for contract recipients.

Vulnerability Details

```
function sendETHToFee(uint256 amount) internal {  
    _marketingWalletAddress.transfer(amount.div(2));  
    _devWalletAddress.transfer(amount.div(2));  
    // charity wallet not included
```

This differs from `swapAndSendToFee`, which distributes to all three wallets.

Impact

- Charity wallet receives nothing when `sendFees()` is used
- `.transfer()` can revert for contract wallets
- Fee distribution becomes inconsistent across code paths

Recommendation

Include charity and use `.call`:

```
(uint256 m, uint256 c, uint256 d) = split(amount);  
(bool ok1,) = _marketingWalletAddress.call{value: m}("");  
(bool ok2,) = _charityWalletAddress.call{value: c}("");  
(bool ok3,) = _devWalletAddress.call{value: d}("");  
require(ok1 && ok2 && ok3, "ETH transfer failed");
```

Status

Resolved

QA

[Q-01] DividendPayingToken.sol#_withdrawDividendOfUser() - Dividend Withdrawn event is emitted before confirming the USDC transfer succeeded

Description

The DividendWithdrawn event is emitted before confirming the USDC transfer. If the transfer fails, the state is rolled back, but the event remains emitted.

Vulnerability Details

```
withdrawnDividends[user] = withdrawnDividends[user].add(_withdrawableDividend);  
emit DividendWithdrawn(user, _withdrawableDividend);  
  
bool success = IERC20(USDC).transfer(user, _withdrawableDividend);  
if (!success) {  
    withdrawnDividends[user] = withdrawnDividends[user].sub(_withdrawableDividend);  
    return 0;  
}
```

The event is emitted even if the transfer fails.

Impact

Off-chain systems record withdrawals that never happened.

Recommendation

Emit after transfer success:

```
bool success = IERC20(USDC).transfer(user, _withdrawableDividend);  
if (!success) {  
    return 0;  
}
```

```
withdrawnDividends[user] += _withdrawableDividend;  
emit DividendWithdrawn(user, _withdrawableDividend);
```

Status

Resolved



[Q-02] Vrine.sol#_transfer() - Max wallet limit check uses the pre-fee amount, rejecting some buys that would be valid post-fee

Description

The max wallet limit check uses the raw transfer amount before fees are deducted. This blocks some valid buys that would be within limits after fees.

Vulnerability Details

Fees are deducted after the limit check in `Vrine.sol#_transfer()` function:

```
require(balanceOf(to) + amount <= maxWalletAmount);  
  
uint256 fees = amount.mul(_totalFees).div(1000);  
  
amount = amount.sub(fees);
```

Impact

Users near the wallet cap can be incorrectly prevented from buying small amounts that would be valid post-fee. Creates unnecessary trading friction.

Recommendation

Apply fee calculation before the check or use net amount.

Status

Resolved

[Q-03] VrineDividendTracker.sol#constructor() - Dividend tracker token name has typo ("Vrine_Dividen_Tracker")

Description

The dividend tracker token name contains a spelling error.

Vulnerability Details

"Dividen" is missing the final "d".

```
constructor()  
  
    DividendPayingToken("Vrine_Dividen_Tracker", "Vrine_Dividend_Tracker")
```

Impact

Cosmetic only, but the name is immutable after deployment and may appear incorrectly in explorers, dashboards and integrations.

Recommendation

Correct to:

```
DividendPayingToken("Vrine_Dividend_Tracker", "Vrine_Dividend_Tracker")
```

Status

Resolved

[Q-04] Magic numbers used throughout code

Description

The dividend tracker token name contains a spelling error.

Vulnerability Details

Several hardcoded values are used without named constants.

```
minimumTokenBalanceForDividends = 100000 * (10**18);  
  
require(newValue >= 200000 && newValue <= 500000, ...);  
  
require(newClaimWait >= 3600 && newClaimWait <= 86400, ...);
```

Impact

Reduces code readability and increases maintenance difficulty.

Recommendation

Define named constants like `MIN_GAS_FOR_PROCESSING`, `MAX_CLAIM_WAIT`, etc.

Status

Resolved

[Q-05] No public getter for `_isBot` Mapping

Description

Users cannot check if an address is blacklisted.

Vulnerability Details

No public function exposes this mapping.

```
mapping (address => bool) private _isBot;
```

Impact

Users get a confusing "Bot." error when trying to trade if blacklisted, with no way to check their status beforehand.

Recommendation

Add a public `isBot(address)` view function.

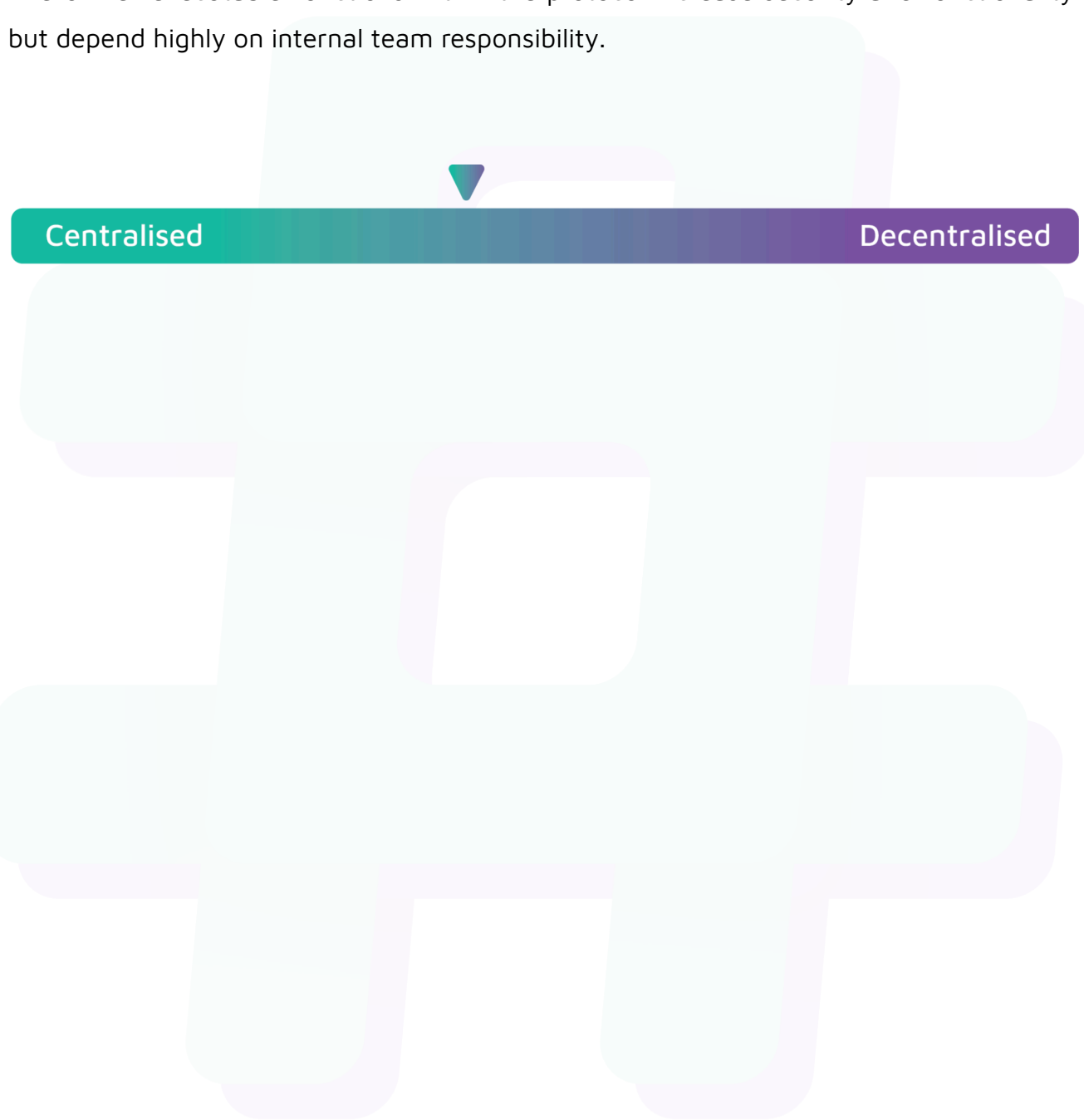
Status

Resolved

Centralisation

The Vrine project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Vrine Network project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.